

SEARCH ALGORITHMS

Techniques used to find a particular element in data

Binary Search

A search algorithm that finds an element in a sorted list

Requires the data to be in sorted order

→ find the middle element of list
→ compare middle element with target
→ If target is small → search in left
→ If target is large → search in right

$O(\log n)$ - logarithmic time

more efficient for large datasets

Linear Search

A search algorithm that finds an element by checking each element from start to end

works on any data (sorted or unsorted)

→ start from the first element
→ compare each element with target
→ If match → element found
→ If not, move to next element

$O(n)$ - linear time

less efficient as data size increases

Definition

Data Requirement

Method

Time complexity

Efficiency

Q46)

DIVIDE AND CONQUER

How it works

- divide - Break the problem
- conquer - solve each subproblem
- Combine - merge

1. Subproblems

original problem is divided into smaller subproblems

Examples

- merge sort
- quick sort
- binary search
- knapsack algorithm

2. Recurrence Relation

$$T(n) = aT(n/b) + f(n)$$

a = no of subproblems

b = division factor

3A parameter 'a'

- indicates how many subproblems are created
- more 'a' means more recursive calls.

3B parameter 'b'

- indicates into how many parts the problem is divided
- Reduces the recursion depth

4. complexity Analysis

Time complexity depends on a, b and $f(n)$. It is analyzed by solving the recurrence.

effect of 'a' & 'b'

If 'a' is large
more work
If 'b' is large
less number of levels

5. Design & Analysis

Design: choose how to divide the problem
Analysis: form recurrence and solve it to find time complexity